

MANUAL DE CONSULTAS

1 • Tabla de Clasificación

```
SELECT
j.nombre_usuario,
j.nivel,
j.region,
MAX(p.puntaje) AS mejor_puntaje
FROM Partidas p
JOIN Jugadores j ON j.jugador_id = p.jugador_id
GROUP BY j.jugador_id, j.nombre_usuario, j.nivel, j.region
ORDER BY mejor_puntaje DESC;
```

¿Qué hace?

Muestra el puntaje más alto que ha logrado cada jugador en toda su historia, junto con su nivel y región.

Solo aparecen jugadores que tienen al menos una partida registrada.

El resultado se ordena de mayor a menor puntaje.

EXPLICACION :

FROM y JOIN: Combina la tabla Partidas con Jugadores mediante el jugador_id. Solo se consideran jugadores que tengan al menos una partida registrada (INNER JOIN).

GROUP BY: Agrupa todos los datos por cada jugador, utilizando su identificador y sus datos personales.

MAX: Dentro de cada grupo (cada jugador), calcula el valor más alto encontrado en la columna puntaje.

ORDER BY: Presenta el listado de forma descendente, colocando al jugador con la puntuación más alta en el primer lugar.

2 • Leaderboard por juego específico

```
SELECT
g.nombre AS juego,
j.nombre_usuario,
p.puntaje,
p.fecha
FROM Partidas p
JOIN Jugadores j ON j.jugador_id = p.jugador_id
JOIN Juegos g ON g.juego_id = p.juego_id
WHERE g.nombre = 'Battle Arena Online'
ORDER BY p.puntaje DESC
LIMIT 10;
```

¿Qué hace?

Lista las 10 mejores puntuaciones obtenidas únicamente en el juego "Battle Arena Online".

Muestra el nombre del juego, el usuario, el puntaje y la fecha.
Está ordenado de mayor a menor puntaje y solo devuelve 10 registros.

EXPLICACION:

JOIN: Enlaza tres tablas. Primero une Partidas con Jugadores para saber el nombre del usuario, y luego con Juegos para obtener el título del juego.

WHERE: Restringe los resultados únicamente a las filas donde el nombre del juego coincida exactamente con 'Battle Arena Online'.

ORDER BY y LIMIT: Ordena los puntajes de mayor a menor y corta la salida para mostrar solo los 10 primeros registros.

3 · Catálogo de items comprados

```
SELECT
    j.nombre_usuario,
    i.nombre AS item,
    i.rareza,
    ii.cantidad,
    ii.fecha_obtencion
FROM Inventario_Items ii
JOIN Jugadores j ON j.jugador_id = ii.jugador_id
JOIN Items i ON i.item_id = ii.item_id
ORDER BY j.nombre_usuario, ii.fecha_obtencion;
```

¿Qué hace?

Muestra todos los objetos (items) que posee cada jugador, incluyendo la rareza, la cantidad que tiene y la fecha en que lo consiguió.

El resultado se ordena alfabéticamente por nombre de usuario y, dentro de cada jugador, de la fecha más antigua a la más reciente

EXPLICACION:

INVENTARIO_ITEMS: Se parte de esta tabla porque actúa como puente entre jugadores y ítems, almacenando además atributos propios de la relación (como cantidad y fecha).

JOIN: Se conecta con Jugadores para obtener el nombre del usuario, y con Items para obtener el nombre del objeto y su rareza.

ORDER BY: Organiza la salida alfabéticamente por el nombre del jugador. Dentro de cada jugador, los objetos se ordenan cronológicamente, desde el más antiguo hasta el más reciente.

4 · Ranking de victorias por jugador

```
SELECT
    j.nombre_usuario,
    SUM(CASE WHEN p.resultado = 'victoria' THEN 1 ELSE 0 END) AS victorias,
    COUNT(p.partida_id) AS partidas_jugadas
FROM Jugadores j
LEFT JOIN Partidas p ON p.jugador_id = j.jugador_id
GROUP BY j.jugador_id, j.nombre_usuario
ORDER BY victorias DESC;
```

Qué hace:

Calcula el total de partidas jugadas y el número de victorias de cada jugador.

A diferencia de las otras, esta consulta usa LEFT JOIN, por lo que también incluye a los jugadores que nunca han jugado (en ese caso, partidas_jugadas y victorias serán 0).

El resultado se ordena de mayor a menor número de victorias.

EXPLICACION:

SUM(CASE WHEN p.resultado = 'victoria' THEN 1 ELSE 0 END) AS victorias): Esta línea calcula las victorias. El CASE evalúa cada fila: si el resultado es 'victoria', devuelve 1; en caso contrario (incluyendo valores NULL), devuelve 0. El SUM suma todos esos valores, dando el total de victorias. El resultado se renombra como victorias.

COUNT(p.partida_id) AS partidas_jugadas): Cuenta el número de partida_id que tiene cada jugador. La función COUNT ignora los NULL, por lo que un jugador sin partidas tendrá 0. Se renombra como partidas_jugadas.

FROM : Define la tabla principal como Jugadores (alias j). Esta es la clave para que aparezcan todos.

LEFT JOIN Realiza una unión hacia la izquierda (LEFT JOIN). Esto significa que se toman TODOS los registros de la tabla de la izquierda (Jugadores) y solo los que coinciden de la tabla de la derecha (Partidas). Si un jugador no tiene partidas, las columnas de Partidas vendrán como NULL, pero el jugador sigue apareciendo.

GROUP BY: Agrupa los resultados por cada jugador (usando su id y nombre) para que las funciones SUM y COUNT se apliquen a cada jugador por separado.

ORDER BY : Ordena el listado final de mayor a menor número de victorias, destacando a los ganadores.